



PUCPR | ARQUITETURA E SOLUÇÕES EM CLOUD

TDE 2

Arquitetura do backend

VERTICAL SLICE · CLEAN ARCHITECTURE · SOLID

Projeto: Veterano. - Plataforma de Mentoria entre Veteranos e Calouros

Repositório: github.com/ViniMTrevisan/pjbl-mentoria-cloud

Branch: tde2-vertical-slice-clean-architecture

Grupo cadastrado: Bento Barp, Guilherme Reis, Guilherme Selenko e Vinicius Trevisan

Data: 05 de outubro de 2026

1. Arquitetura e escopo

A aplicação foi reorganizada por caso de uso, preservando as rotas e as fontes de dados atuais. O frontend continua consumindo Azure Functions; o catálogo de mentores e as sessões permanecem mock, enquanto o CRUD de mentores continua no MongoDB Atlas.

Direção das dependências

Azure Functions -> fatia de aplicação -> domínio/porta -> adaptador. A composição ocorre na borda em `functions/dependencies.js`. Regras de negócio não importam o SDK Azure nem o driver MongoDB.

Área	Local	Responsabilidade
Domínio	<code>api/src/domain/mentor.js</code>	Validação, reputação e normalização.
Fatias	<code>api/src/features/</code>	Um módulo por operação de mentores e sessões.
Porta	<code>api/src/ports/MentorRepository.js</code>	Contrato estrutural das operações de persistência.
Adaptadores	<code>api/src/infrastructure/</code>	Catálogo mock, repositório Mongo e conexão reutilizável.
Entrada	<code>api/src/functions/</code>	Bindings HTTP, parsing, composição e resposta JSON.

Rotas preservadas

Rota	Fatia	Fonte
GET /api/mentores	<code>listarMentoresPublicos</code>	Mock
GET /api/mentores/{id}	<code>obterMentorPublico</code>	Mock
GET /api/sesoes?status=	<code>listarSesoes</code>	Mock
GET /api/health	<code>health check</code>	Azure Functions
POST /api/mentores-db	<code>criarMentor</code>	MongoDB
GET /api/mentores-db?q=&id=	<code>pesquisarMentores</code>	MongoDB
PUT PATCH /api/mentores-db/{id}	<code>alterarMentor</code>	MongoDB
DELETE /api/mentores-db/{id}	<code>excluirMentor</code>	MongoDB

2. Aplicação de SOLID

Princípio	Aplicação no backend
S - Responsabilidade única	Handler trata HTTP; cada fatia coordena uma operação; domínio concentra regra; MongoRepository persiste.
O - Aberto/fechado	As fatias recebem dependências por contrato e permitem conectar outro adaptador sem reescrever o handler.
L - Substituição de Liskov	Um repositório pode substituir outro ao cumprir os métodos e resultados da porta MentorRepository.
I - Segregação de interfaces	Catálogo mock atende leituras públicas; o contrato do repositório cobre apenas o CRUD Mongo.
D - Inversão de dependência	Casos de uso dependem de portas e recebem adaptadores concretos montados na borda.

Validações e falhas preservadas

Corpo ausente ou inválido, campos obrigatórios e período fora de 1-12 retornam 400. Update vazio também retorna 400; ID inexistente retorna 404. A busca continua escapando metacaracteres de regex e limita a 100 registros. A alteração persiste por operação atômica \$set. Erros técnicos são registrados no servidor e a resposta 500 não expõe detalhes do driver.

Não foram alterados autenticação, frontend, configuração Azure ou serviços publicados. O nível anonymous existente e o comportamento de sessões com datas ISO foram mantidos.

Alunos e contribuições propostas

O repositório registra quatro integrantes, mas não registra a divisão individual desta etapa. Esta proposta deve ser validada pelo grupo antes de ser apresentada como contribuição realizada.

Aluno	Participação sugerida para validação
Bento Barp	Revisar regras de mentores/sessões e contratos HTTP preservados.
Guilherme Reis	Revisar CRUD, porta de repositório e persistência MongoDB.
Guilherme Selenko	Revisar diagramas e aplicação de Clean Architecture, Vertical Slice e SOLID.
Vinicius Trevisan	Integrar a mudança à branch e consolidar documentação e entrega.

A implementação foi assistida por IA generativa; a tabela não comprova participação individual sem validação dos alunos.

3. Diagrama de classes e módulos

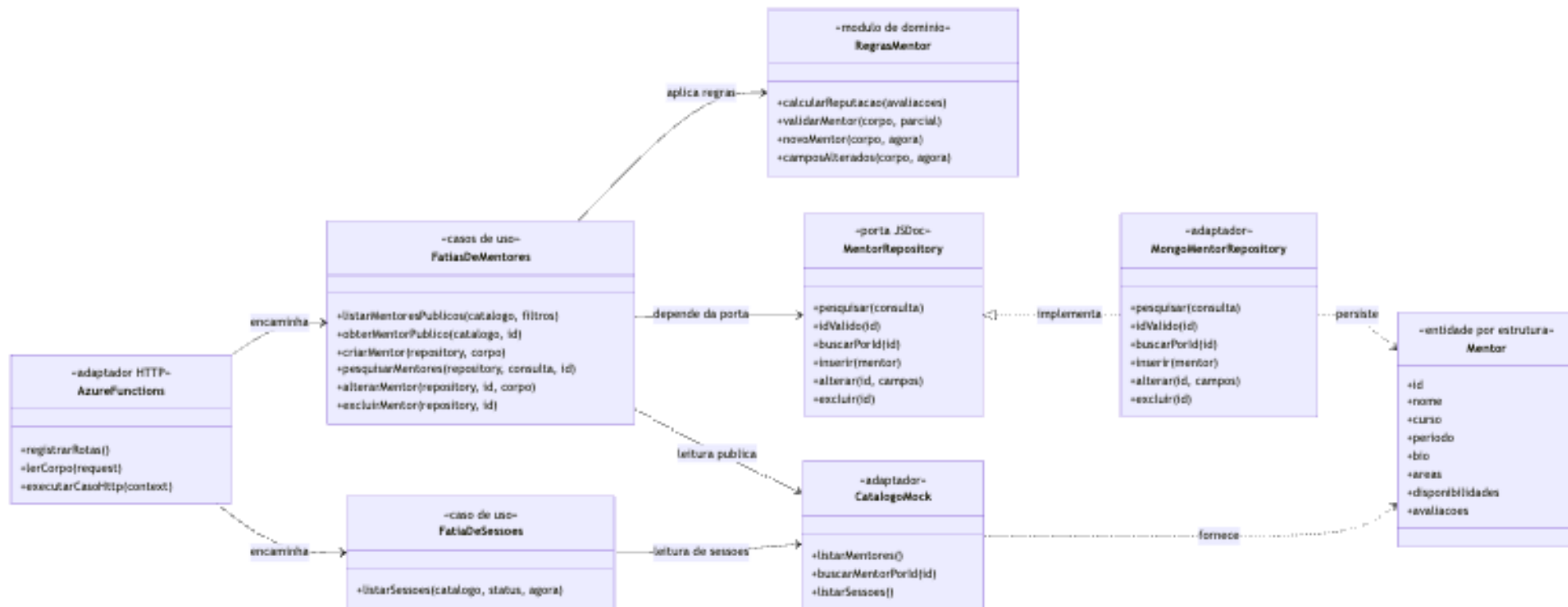


Figura 1. Entidade por estrutura, regras puras, contrato do repositório e módulos que coordenam as fatias. Em JavaScript, esses elementos são funções e objetos; o diagrama expressa o modelo UML conceitual.

4. Diagrama de componentes do backend

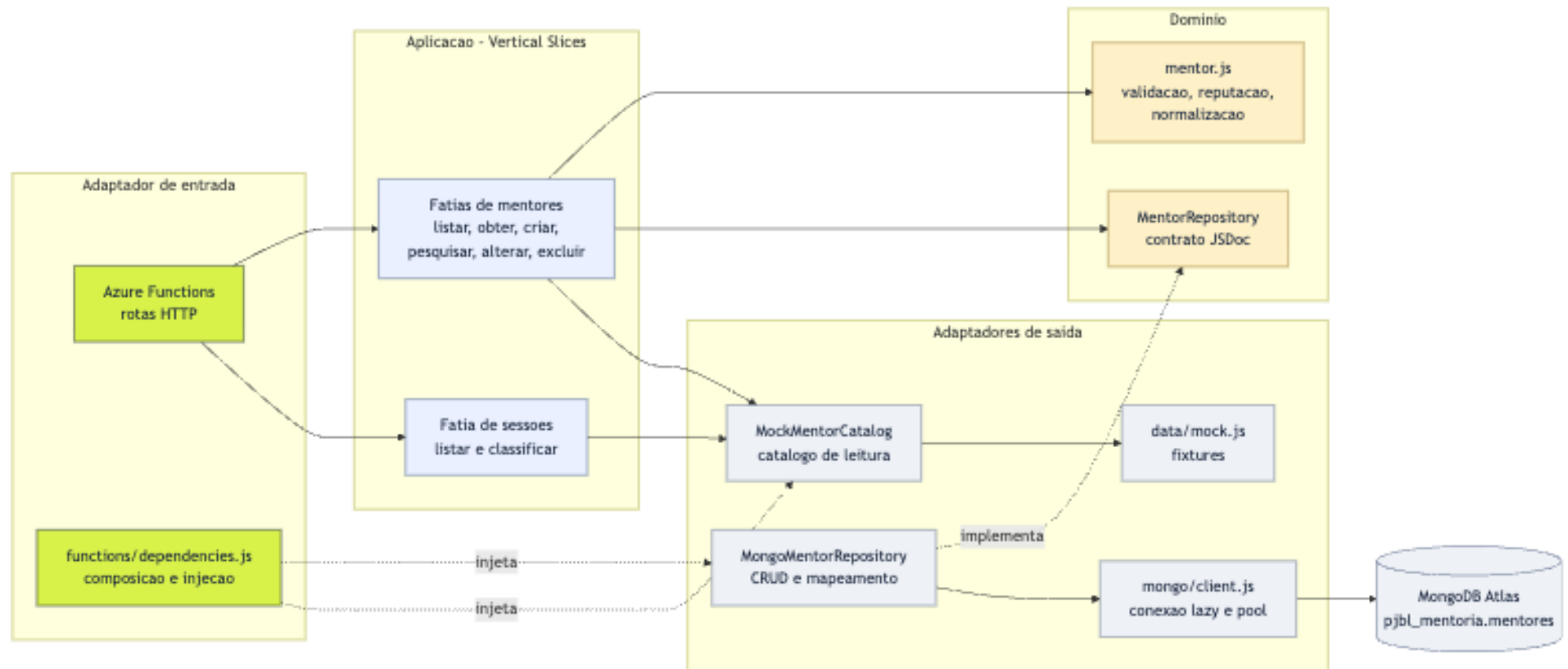


Figura 2. Azure Functions encaminha requisições às fatias; catálogo mock e repositório Mongo são adaptadores de saída distintos.

5. Prompts utilizados

A lista completa inclui o pedido base do aluno e os prompts técnicos gerados para orientar a refatoração, os diagramas, a documentação e a revisão final.

Prompt 1 - pedido base

Utilizar IA Generativa para aplicar o VERTICAL SLICE e CLEAN ARCHITECTURE e SOLID no backend da aplicação TDE 2.

Utilizar o código da aplicação que está sendo criada no TDE 2.

Em documento PDF entregar:

No arquivo deve constar o nome dos ALUNOS que auxiliou na tarefa, independente de ser atividade em grupo. (Descreva o que cada aluno realizou).

Informar o GITHUB do projeto em uma nova branch.

Informar todos os prompts utilizados para modificar a aplicação.

Entregar diagrama de classes e componentes do BACKEND da aplicação em VERTICAL SLICE e CLEAN ARCHITECTURE e SOLID (gerar em markdown e imagem).

(gere os prompts vc mesmo e faça tudo)

Prompt 2 - análise e refatoração do backend (gerado)

Inspecione o backend existente da plataforma Veterano. e aplique Vertical Slice, Clean Architecture e SOLID sem reescrever o produto. Preserve as rotas, métodos, formatos de sucesso e status usados pelo frontend. Mantenha o catálogo público e as sessões em dados mock, e mantenha o CRUD de mentores no MongoDB.

Deixe os handlers Azure Functions como adaptadores HTTP finos. Separe cada caso de uso em sua própria fatia, extraia regras puras de mentor para o domínio, descreva a porta de repositório e mantenha MongoDB e mock em adaptadores de infraestrutura. Monte os adaptadores na borda e injete suas dependências nas fatias.

Preserve as validações, o escape de regex, o limite de 100 registros, a atualização parcial atômica e os status 400/404 existentes. Erros internos do MongoDB devem ser registrados no servidor e não expostos ao cliente. Não mude frontend, autenticação, configuração Azure nem faça deploy.

5. Prompts utilizados (continuação)

Prompt 3 - diagramas e documentação de arquitetura (gerado)

Com base somente na implementação final do backend, documente as fatias verticais, as fronteiras da Clean Architecture, os princípios SOLID e o mapeamento de cada endpoint ao caso de uso e ao adaptador. Gere os diagramas Mermaid de classes/módulos e componentes em arquivos Markdown editáveis e renderize cada diagrama como SVG.

Use nomes que correspondam aos arquivos e funções do código. Mostre a direção das dependências e diferencie catálogo mock de persistência MongoDB. Se o diagrama de classes representar funções/objetos estruturais do JavaScript, deixe isso explícito.

Prompt 4 - relatório e revisão da entrega (gerado)

Monte um PDF acadêmico em português com objetivo, escopo, arquitetura aplicada, princípios SOLID, endpoints preservados, diagramas de classes e componentes, repositório e branch do GitHub, prompts completos e identificação dos integrantes do grupo. Marque contribuições individuais como proposta para validação quando não houver evidência no repositório; não atribua atividades como fatos sem confirmação.

Revise o código e os artefatos contra o pedido original. Confirme sintaxe, referências dos diagramas, texto extraído e renderização visual de todas as páginas. Relate separadamente o que foi validado localmente e o que não foi validado no Azure ou MongoDB.

GitHub

github.com/ViniMTrevisan/pjbl-mentoria-cloud/tree/tde2-vertical-slice-clean-architecture

Verificação

Os arquivos JavaScript passaram por verificação sintática com `node --check`. Os diagramas foram renderizados em SVG e PNG e inspecionados. O PDF foi renderizado em imagens para conferência de layout. Não houve validação de integração contra MongoDB, Azure ou ambiente publicado.